Name: Anchal Singh
Roll No: 119

PRACTICAL NO : 4

Introduction to Python Programming : Learn the different Libraries- NumPy, Pandas, SciPy , Matplotlib , Scikit Learn

Introduction to Python Programming

[1]:
```
print("Anchal Singh 119")
import numpy as np

# Create a 1D array
a = np.array([1, 2, 3, 4, 5])

print("1D Array:", a)
print("Type:", type(a))
print("dtype:", a.dtype)
print("Shape:", a.shape)

# Create a 2D array
b = np.array([[1, 2, 3], [4, 5, 6]])

print("\n2D Array:\n", b)
print("Shape:", b.shape)

# Basic operations
print("\nMean:", np.mean(a))
print("Sum:", np.sum(a))
print("Max:", np.max(a))
print("Min:", np.min(a))
```

```
Anchal Singh 119
1D Array: [1 2 3 4 5]
Type: <class 'numpy.ndarray'>
dtype: int64
Shape: (5,)

2D Array:
 [[1 2 3]
 [4 5 6]]
Shape: (2, 3)

Mean: 3.0
Sum: 15
Max: 5
Min: 1
```

Creating NumPy Array + Checking Type

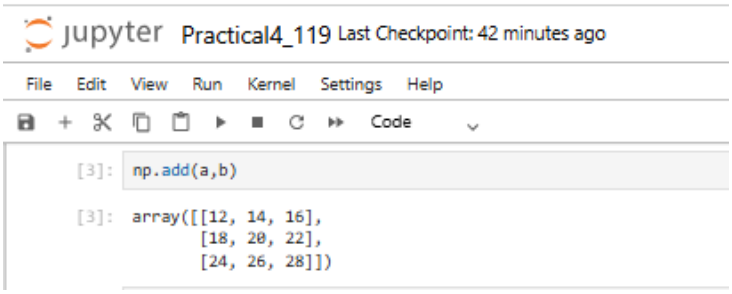
1. **Create Base Arrays.**

[2]:
```
import numpy as np

# Anchal Singh 119

a = np.array([[1, 2, 3],
              [4, 5, 6],
              [7, 8, 9]])

b = np.arange(start=11, stop=20).reshape(3, 3)

print(a)
print(b)
```

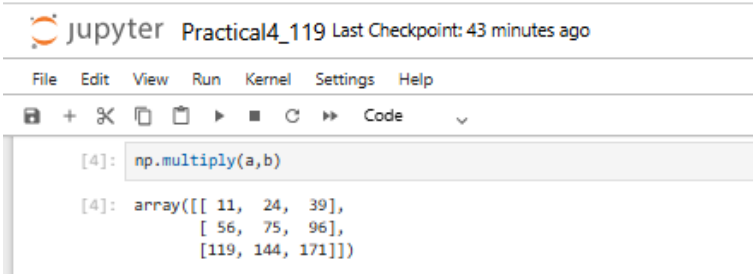
```
[[1 2 3]
 [4 5 6]
 [7 8 9]]
[[11 12 13]
 [14 15 16]
 [17 18 19]]
```

Name: Anchal Singh
Roll No: 119

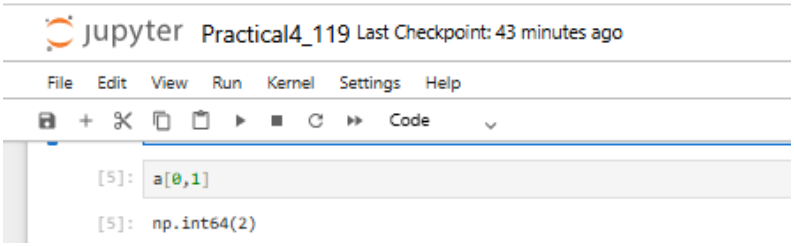
2. **NumPy Addition (Element-wise)**



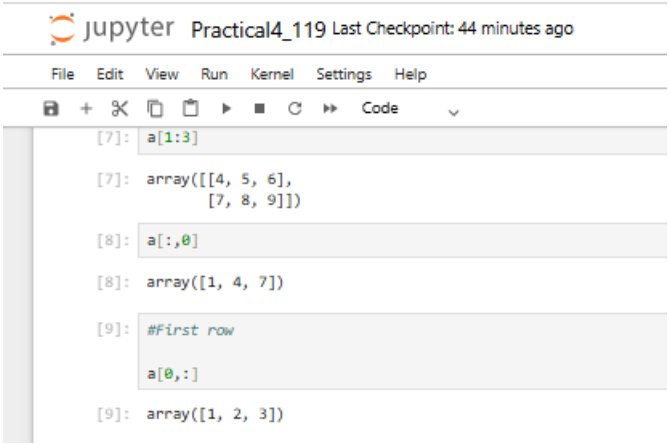
3. **NumPy Multiplication (Element-wise)**



4. **Accessing Elements (Indexing)**

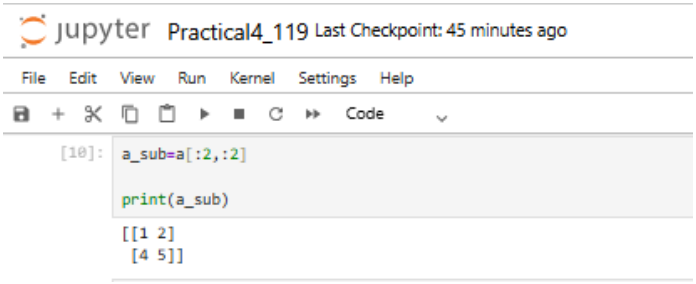


5. **Accessing Rows & Columns (Slicing)**

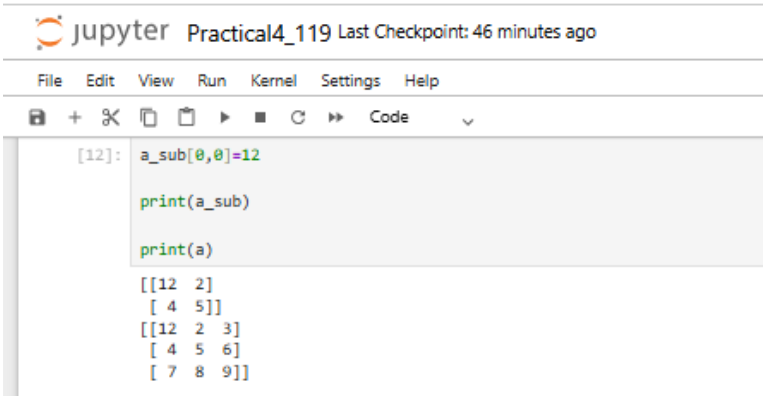


Name: Anchal Singh
Roll No: 119

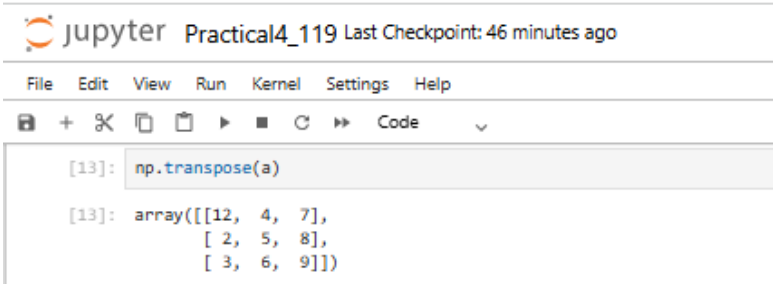
6. **Subset of Array (Submatrix)**



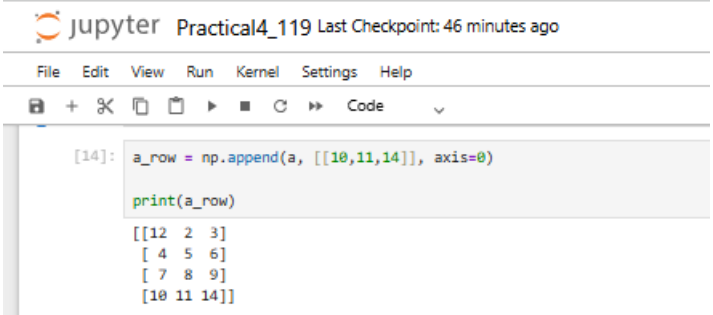
7. **Modify Subset (Affects Original Array)**



8. **Transpose of Array**

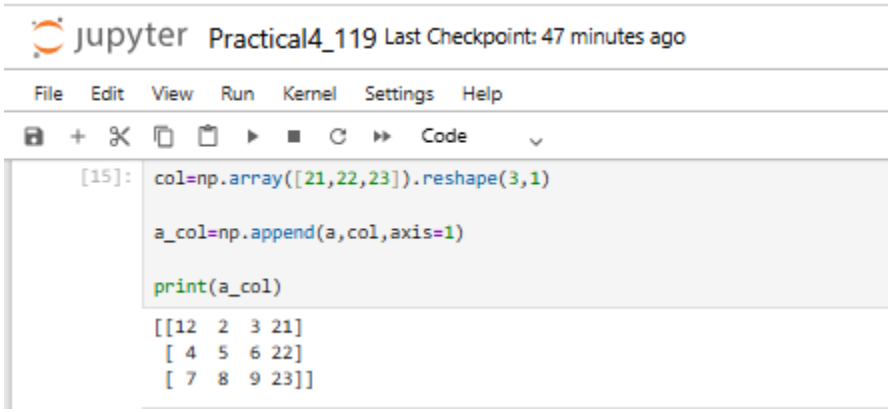


9. **Append Row**

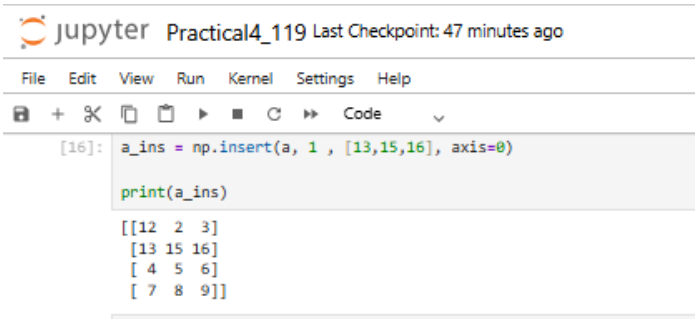


Name: Anchal Singh
Roll No: 119

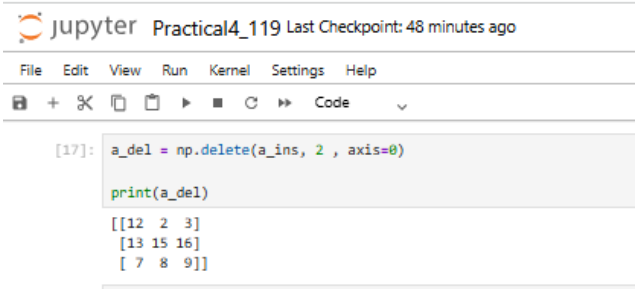
10. Append Column



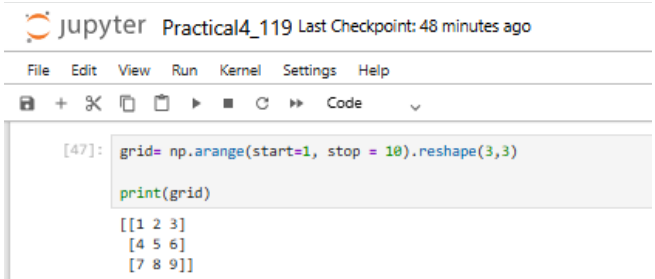
11. Insert Row at Specific Position



12. Delete Row

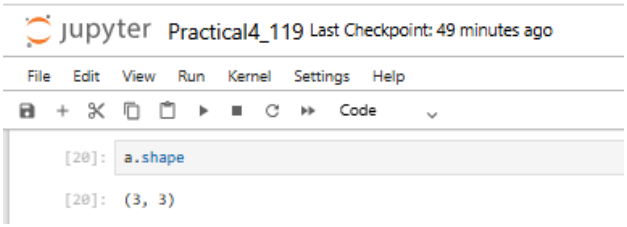


13. Reshape Array

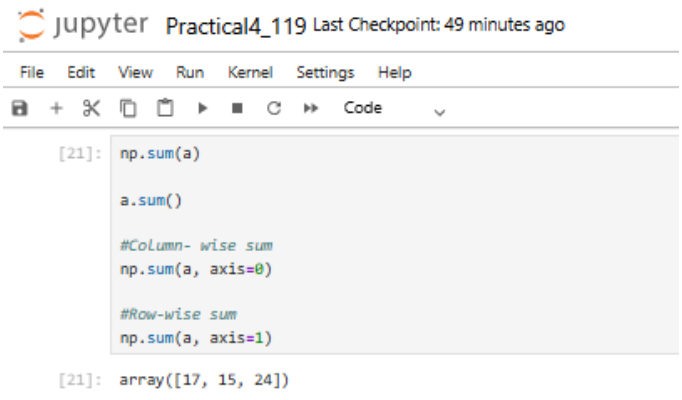


Name: Anchal Singh
Roll No: 119

14. Shape of Array



15. Sum of Array (Different Axes)



Name: Anchal Singh
Roll No: 119

Pandas

```
[23]: import pandas as pd

cars_data1= pd.read_csv(r"C:\Users\anchsingh\Anchal Singh\Data Analysis with Python\AIML\119_Anchal_Singh\mtcars.csv")

cars_data1.head()
```

[23]:

	model	mpg	cyl	disp	hp	drat	wt	qsec	vs	am	gear	carb
0	Mazda RX4	21.0	6	160.0	110	3.90	2.620	16.46	0	1	4	4
1	Mazda RX4 Wag	21.0	6	160.0	110	3.90	2.875	17.02	0	1	4	4
2	Datsun 710	22.8	4	108.0	93	3.85	2.320	18.61	1	1	4	1
3	Hornet 4 Drive	21.4	6	258.0	110	3.08	3.215	19.44	1	0	3	1
4	Hornet Sportabout	18.7	8	360.0	175	3.15	3.440	17.02	0	0	3	2

```
[26]: cars_data1['model']= cars_data1['model'].astype('object')
```

```
[27]: cars_data1.dtypes
```

```
[27]: model     object
      mpg      float64
      cyl        int64
      disp     float64
      hp         int64
      drat     float64
      wt       float64
      qsec     float64
      vs         int64
      am         int64
      gear       int64
      carb       int64
      dtype: object
```

1. Count of Unique Data Types

```
[28]: cars_data1.dtypes.value_counts()
```

```
[28]: int64      6
      float64    5
      object     1
      Name: count, dtype: int64
```

2. Selecting Data Based on Data Types

Name: Anchal Singh
Roll No: 119

[29]:
```
cars_data1.select_dtypes(exclude=["object"])
```

[29]:

	mpg	cyl	disp	hp	drat	wt	qsec	vs	am	gear	carb
0	21.0	6	160.0	110	3.90	2.620	16.46	0	1	4	4
1	21.0	6	160.0	110	3.90	2.875	17.02	0	1	4	4
2	22.8	4	108.0	93	3.85	2.320	18.61	1	1	4	1
3	21.4	6	258.0	110	3.08	3.215	19.44	1	0	3	1
4	18.7	8	360.0	175	3.15	3.440	17.02	0	0	3	2
5	18.1	6	225.0	105	2.76	3.460	20.22	1	0	3	1
6	14.3	8	360.0	245	3.21	3.570	15.84	0	0	3	4
7	24.4	4	146.7	62	3.69	3.190	20.00	1	0	4	2
8	22.8	4	140.8	95	3.92	3.150	22.90	1	0	4	2
9	19.2	6	167.6	123	3.92	3.440	18.30	1	0	4	4
10	17.8	6	167.6	123	3.92	3.440	18.90	1	0	4	4
11	16.4	8	275.8	180	3.07	4.070	17.40	0	0	3	3
12	17.3	8	275.8	180	3.07	3.730	17.60	0	0	3	3
13	15.2	8	275.8	180	3.07	3.780	18.00	0	0	3	3
14	10.4	8	472.0	205	2.93	5.250	17.98	0	0	3	4
15	10.4	8	460.0	215	3.00	5.424	17.82	0	0	3	4
16	14.7	8	440.0	230	3.23	5.345	17.42	0	0	3	4
17	32.4	4	78.7	66	4.08	2.200	19.47	1	1	4	1
18	30.4	4	75.7	52	4.93	1.615	18.52	1	1	4	2
19	33.9	4	71.1	65	4.22	1.835	19.90	1	1	4	1
20	21.5	4	120.1	97	3.70	2.465	20.01	1	0	3	1
21	15.5	8	318.0	150	2.76	3.520	16.87	0	0	3	2
22	15.2	8	304.0	150	3.15	3.435	17.30	0	0	3	2

3. Summary of DataFrame

[30]:
```
cars_data1.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 32 entries, 0 to 31
Data columns (total 12 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   model   32 non-null     object
 1   mpg     32 non-null     float64
 2   cyl     32 non-null     int64
 3   disp    32 non-null     float64
 4   hp      32 non-null     int64
 5   drat    32 non-null     float64
 6   wt      32 non-null     float64
 7   qsec    32 non-null     float64
 8   vs      32 non-null     int64
 9   am      32 non-null     int64
 10  gear    32 non-null     int64
 11  carb    32 non-null     int64
dtypes: float64(5), int64(6), object(1)
memory usage: 3.1+ KB
```

Name: Anchal Singh
Roll No: 119

4. Unique Values of a Column (mpg)

[31]:
```
import numpy as np

print(np.unique(cars_data1['mpg']))
```

```
[10.4 13.3 14.3 14.7 15.  15.2 15.5 15.8 16.4 17.3 17.8 18.1 18.7 19.2
 19.7 21.  21.4 21.5 22.8 24.4 26.  27.3 30.4 32.4 33.9]
```

5. Unique Values of HP

[32]:
```
print(np.unique(cars_data1['hp']))
```

```
[ 52  62  65  66  91  93  95  97 105 109 110 113 123 150 175 180 205 215
 230 245 264 335]
```

6. Unique Values of gear

[33]:
```
print(np.unique(cars_data1['gear']))
```

```
[3 4 5]
```

[48]:
```
print("Anchal Singh 119")
import pandas as pd
# Create a DataFrame
data = {
    "Name": ["Prince", "Bob", "Charlie", "David"],
    "Age": [25, 30, 35, 28],
    "Score": [85.5, 90.0, 78.3, 92.1]
}
df = pd.DataFrame(data)
print("DataFrame:")
print(df)
# Info and description
print("\nInfo:")
df.info()
print("\nDescribe:")
print(df.describe())
# Select a column
print("\nSelect column Name:")
print(df["Name"])
```

Name: Anchal Singh
Roll No: 119

```
Anchal Singh 119
DataFrame:
      Name  Age  Score
0   Prince   25   85.5
1      Bob   30   90.0
2  Charlie   35   78.3
3    David   28   92.1

Info:
<class 'pandas.DataFrame'>
RangeIndex: 4 entries, 0 to 3
Data columns (total 3 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   Name    4 non-null      str
 1   Age     4 non-null      int64
 2   Score   4 non-null      float64
dtypes: float64(1), int64(1), str(1)
memory usage: 228.0 bytes

Describe:
             Age     Score
count   4.000000   4.00000
mean   29.500000  86.47500
std     4.203173   6.10594
min    25.000000  78.30000
25%    27.250000  83.70000
50%    29.000000  87.75000
75%    31.250000  90.52500
max    35.000000  92.10000

Select column Name:
0     Prince
1        Bob
2    Charlie
3      David
Name: Name, dtype: str
```

Name: Anchal Singh
Roll No: 119

Scipy

[35]:
```
print("Anchal Singh 119")

from scipy import stats
import numpy as np

data = np.array([2, 4, 4, 5, 5, 7, 9])

# Basic statistics
print("Mean:", np.mean(data))
print("Std Dev:", np.std(data))

# One-sample t-test
t_stat, p_value = stats.ttest_1samp(data, popmean=5)

print("t-statistic:", round(t_stat, 4))
print("p-value:", round(p_value, 4))

# Normal distribution
norm_dist = stats.norm(loc=0, scale=1)

print("\nNormal Distribution (mean=0, std=1):")
print("PDF at x=0:", round(norm_dist.pdf(0), 4))
print("CDF at x=1:", round(norm_dist.cdf(1), 4))
```

```
Anchal Singh 119
Mean: 5.142857142857143
Std Dev: 2.099562636671296
t-statistic: 0.1667
p-value: 0.8731

Normal Distribution (mean=0, std=1):
PDF at x=0: 0.3989
CDF at x=1: 0.8413
```

Name: Anchal Singh
Roll No: 119

Scikit Learn

[36]:
```python
print("Anchal Singh 119")

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report

# Load dataset
iris = load_iris()

X, y = iris.data, iris.target

print("Dataset Shape:", X.shape)
print("Classes:", iris.target_names)

# Train Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Feature Scaling
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Model Training
model = LogisticRegression(max_iter=200)

model.fit(X_train, y_train)

# Prediction and Accuracy
y_pred = model.predict(X_test)

print("\nAccuracy:", accuracy_score(y_test, y_pred))

print("\nClassification Report:")
print(classification_report(y_test, y_pred, target_names=iris.target_names))
```

```
Anchal Singh 119
Dataset Shape: (150, 4)
Classes: ['setosa' 'versicolor' 'virginica']

Accuracy: 1.0

Classification Report:
              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        10
  versicolor       1.00      1.00      1.00         9
   virginica       1.00      1.00      1.00        11

    accuracy                           1.00        30
   macro avg       1.00      1.00      1.00        30
weighted avg       1.00      1.00      1.00        30
```

Name: Anchal Singh
Roll No: 119

[38]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

credit_df = pd.read_csv(r"C:\Users\anchsingh\Anchal Singh\Data Analysis with Python\AIML\119_Anchal_Singh\Salary_Data.csv")

credit_df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 30 entries, 0 to 29
Data columns (total 2 columns):
 #   Column           Non-Null Count  Dtype  
---  ------           --------------  -----  
 0   YearsExperience  30 non-null     float64
 1   Salary           30 non-null     int64  
dtypes: float64(1), int64(1)
memory usage: 612.0 bytes
```

[39]:
```
credit_df.head()
```

[39]:

	YearsExperience	Salary
0	1.1	39343
1	1.3	46205
2	1.5	37731
3	2.0	43525
4	2.2	39891

[40]:
```
credit_df.size
```

[40]:
```
60
```

[41]:
```
credit_df.shape
```

[41]:
```
(30, 2)
```

[42]:
```
credit_df.describe()
```

[42]:

	YearsExperience	Salary
count	30.000000	30.000000
mean	5.313333	76003.000000
std	2.837888	27414.429785
min	1.100000	37731.000000
25%	3.200000	56720.750000
50%	4.700000	65237.000000
75%	7.700000	100544.750000
max	10.500000	122391.000000

Name: Anchal Singh
Roll No: 119

[43]:
```
X= credit_df['YearsExperience']

X.head()
```

[43]:
```
0    1.1
1    1.3
2    1.5
3    2.0
4    2.2
Name: YearsExperience, dtype: float64
```

[45]:
```
Y= credit_df['Salary']

Y.head()
```

[45]:
```
0    39343
1    46205
2    37731
3    43525
4    39891
Name: Salary, dtype: int64
```

[46]:
```
plt.figure(figsize=(6,4))

sns.pairplot(credit_df, x_vars= ['YearsExperience'], y_vars=['Salary'], size=4, kind='scatter')

plt.xlabel('Years')

plt.ylabel('Salary')

plt.title('Salary Prediction')

plt.show()
```

```
C:\Users\anchsingh\AppData\Local\Programs\Python\Python314\Lib\site-packages\seaborn\axisgrid.py:2100: UserWa
ed to `height`; please update your code.
  warnings.warn(msg, UserWarning)
<Figure size 600x400 with 0 Axes>
```